

Progress Report

- Increment 1 -

Group #1

1) Team Members

Zachary Lima | zol21 | zlima410

Dylan Connolly | dtc22h | dylantcon

Ivan Lepesii | il23d | Kupinaaa

Colby Leavitt | cal22b | Colby-Leavitt

2) Project Title and Description

Project Title: LibMaster

Description:

We are revamping the FSU library study room reservation system to be more accessible, easier to use, and more visually appealing. We are making it easier to visualize which rooms are available and which are not. We are adding better-querying capabilities and allowing users to sort the available rooms by library, floor, size, and time available. We will add a visualization of which rooms are available in a floor-map format.

3) Accomplishments and overall project status during this increment

Most of the backend functionality has been completed such as the models and serializers used to implement with the frontend. The frontend is less flushed out and mainly just has enough to demo in the video. Our current goal is to get to a minimum viable product that we can expand off of to keep the project from getting disorganized.

4) Challenges, changes in the plan and scope of the project and things that went wrong during this increment

One particularly time-consuming challenge was the absence of the option to set the refresh token as an HTTP-only cookie in `django-rest-framework-simplejwt` library, which introduces a security vulnerability. To solve this, we had to go through the github of the library and find the workaround posted on github by the author of the library.

Additionally, another challenge that presented itself was constructing a comprehensive model system for the Library, Floor, Room, and Reservation models. The relationship between these models created

a complex hierarchical structure that required careful validation. Each model had to be checked not only against its own constraints, but also against parent entity constraints. My solution was to implement a multi-layered validation system that cascaded checks throughout the hierarchy. For example, the Reservation model's `clean()` method examines not just reservation times, but also checks against the parent Room's availability, which in turn depends on its Floor's status, and ultimately the Library's operating hours and constraints.

Validating time-based constraints proved particularly difficult, especially preventing overlapping reservations while handling status changes. I developed a specialized validation approach in the Reservation model that:

- Uses Django's Q objects for complex queries for conflicting reservations
- Implements time range checks to ensure reservations fall within library hours
- Adds a database-level constraint (`check_end_time_after_start_time`) to enforce time ordering even outside the Python validation
- Differentiates between new reservations and updates to existing ones

Yet another significant challenge was addressing race conditions where two users might try to reserve the same room at the same time, leading to potential double bookings. To fix this, I implemented a combination of optimistic concurrency control through Django's transaction system and rigorous validation at multiple levels:

- Model-level validation through `clean()` methods
- Database constraints for fundamental rules
- Additional validation logic in serializers for the API layer
- Status-based filtering to prevent conflicting actions

5) Team Member Contribution for this increment

a) Progress Report Contributions:

- Dylan:** In the Progress Report, I described the challenges encountered while architecting the backend models in Django. I also recorded the demonstration of the core Django models powering the application.
- Ivan:** Made the video including editing the parts recorded by other team members
- Zach:** Wrote the project description as well as the accomplishments and overall project status.
- Colby:** Helped with project title and description, challenges, and wrote the stakeholder communication email.

b) R&D Document Contributions:

- Dylan:** I worked on several elements in the Requirements and Design document. I authored the Overview, Functional Requirements, Non-Functional Requirements, Operating Environment, and Assumptions and Dependencies sections. I also created the Use-Case UML Diagram using the `draw.io` tool.
- Ivan:** Helped writing the requirements and Operating Environment
- Zach:** Created the class and sequence diagrams.

- iv) **Colby:** Helped with the use case diagram.

c) I&T Document Contributions:

- i) **Dylan:** Assisted in writing the Programming Languages section.
- ii) **Ivan:** Helped with the technologies used and execution based functional testing
- iii) **Zach:** Helped with the programming languages section as well as the technologies section.
- iv) **Colby:** Assisted in writing sections 3 (student functional requirements testing), 4 (non-functional requirements testing), and 5 (non-functional testing section).

d) Source Code Contributions:

- i) **Dylan:** I configured the frontend architecture's main stack, including its directory structure and a linter for TypeScript ('eslint'). I also assisted in implementing the backend Django configuration, set up our PostgreSQL integration, and the cross-origin resource sharing headers for communication between the frontend and backend. After this, I designed, implemented, and tested the Library, Floor, Room, and Reservation Models that power the application's core functionality.
- ii) **Ivan:** Implemented a secure JWT authentication and authorization system. With 'djangoframeworksimple-jwt' on the backend and a custom react auth context and useAuth hook available across the frontend. As well as setting up react-router + private routes.
- iii) **Zach:** Implemented a few of the serializers for the backend to communicate with the frontend using JSON. Also created the context and library browser pages on the frontend to implement the functionality for our backend routes.
- iv) **Colby:** Implemented Materials class, including backend database implementation and display functionality in the front end. Also worked on front end API endpoints and url setup in conjunction with Zach.

6) Plans for the next increment

For the next increment, we will begin to implement the frontend for reservations that are currently open as well as the reservations per floor or library. We hope to begin working on the floor mapping system as well, to make reservations more visually appealing.

7) Stakeholder Communication

Dear Stakeholders,

I hope you are doing well. I am excited to share our development team's progress on Libmaster. We have made significant progress in both the front-end and back-end development of the system while addressing challenges that have arisen during the development process.

We have implemented a number of core features, including: a database to store library and user information, user authentication, and a simple user interface. At this time, regular users can log

into their account or create a new one, and can view and select libraries, floors, rooms, and study supplies. Initial testing has gone smoothly thus far.

Some challenges we encountered and overcame included: issues implementing authentication tokens, which required a creative workaround provided in the github of the library itself; constructing a comprehensive model system for the Library, Floor, Room, and Reservation models, which we solved by implementing a multi-layered validation system that cascaded checks throughout the hierarchy.

In our next round of design implementations, we plan to implement a functional system that allows users to reserve rooms. We also plan to implement a map-based display of rooms in libraries, giving users a visually appealing way to view room availability.

We appreciate your continued support and insights as we move forward. Please let us know if you have any specific questions or input at this stage. We look forward to sharing more progress in our next update.

Best regards,

Colby Leavitt [Libmaster development team]

8) Link to video

<https://www.youtube.com/watch?v=veZ7ylGb5Cw>